

NEM = Node Evolution Molecule

节点进化分子 v0.1

AEP 原子化工程体系概念锁定文档

版本	v0.1
日期	2026-06-02
定位	AI 原生工程体系中的节点分子层概念
核心命题	AEP 是工程原子，NEM 是节点进化分子

0. 一句话定义

NEM (Node Evolution Molecule, 节点进化分子) 是由多个 AEP 工程原子组合形成的可进化工程模块。

NEM 不是普通意义上的“节点”或“任务集合”，而是 AI 原生工程体系中的分子级能力单元：它可以通过不断吸收新的 AEP 完成基础构建、能力增强、稳定化、重构与版本升级，并作为独立模块被其他工程链依赖、调用、替换或复用。

用最短的结构表达：

AEP = Engineering Atom / 工程原子
NEM = Node Evolution Molecule / 节点进化分子
Track = Engineering Chain / 工程链
Project = Engineering Organism / 工程生命体

1. 为什么需要 NEM 这个新概念

在传统项目管理里，“节点”通常只是计划表上的一个阶段、一个里程碑、一个任务集合。这个概念在 AI 时代不够用了。

因为 AI 原生工程不是靠一个人连续推进，而是靠大量标准化工程工作单元被反复投放、执行、验证、复盘和升级。一个“节点”不再是静态的点，而更像一个不断生长的模块。

过去的节点：

节点 = 计划中的一个阶段
完成 = 做完这个阶段

AI 原生工程中的 NEM：

NEM = 由多个 AEP 工程原子组合形成的可进化模块
完成 = 达到某个版本的成熟度，而不是永久结束

这意味着我们不再说“节点完成了”，而是说：

这个 NEM 达到 v0.1 BASIC。
这个 NEM 升级到 v0.3 STABLE。
这个 NEM 进入 v1.0 ADOPTED。
这个 NEM 需要 v2.0 REFACTORED。

NEM 的提出，是为了把“节点”从普通项目管理概念，升级为 AI 时代的模块化工程生命单元。

2. NEM 的正式定义

2.1 英文定义

NEM (Node Evolution Molecule) is an evolvable engineering module composed of multiple AEP engineering atoms. It represents a node-level capability unit that can be versioned, upgraded, validated, refactored, depended upon, and reused across AI-native engineering systems.

2.2 中文定义

NEM（节点进化分子）是由多个 AEP 工程原子组成的可进化工程模块。

一个 NEM 不代表一次性完成任务，而代表一个可以持续升级、版本化、重构、被依赖、被替换和被复用的工程能力单元。

2.3 概念边界

NEM 不是：

- 不是普通任务清单；
- 不是一次性工单；
- 不是单个 Markdown 文件；
- 不是单个 AEP；
- 不是只存在于项目计划表里的抽象节点；
- 不是完成后永远不再变化的静态模块。

NEM 是：

- 一个工程分子；
- 一个节点级模块；
- 一个可持续吸收 AEP 的进化结构；
- 一个可以被下游模块依赖的工程能力；
- 一个通过 PoEW 和 Gate 证明其成熟度的版本化模块。

3. AEP 与 NEM 的关系

AEP 与 NEM 的关系可以用“原子 - 分子”解释。

AEP 是工程原子。

NEM 是由多个工程原子组成的工程分子。

一个 AEP 只能完成一个明确的工程做功目标。例如：

- AEP-001: 建立 Runtime 基础协议
- AEP-002: 执行 90-task full test
- AEP-003: 增加状态字段系统
- AEP-004: 增加失败分类系统
- AEP-005: 生成 final summary

这些 AEP 组合起来，就可以让一个 Runtime NEM 达到基础可用状态。

因此：

- AEP 负责做功。
- NEM 负责积累能力。
- AEP 产生 PoEW。
- NEM 通过 Gate 升级版本。

4. NEM 的核心公式

可以用一个抽象公式描述 NEM:

$$NEM_v = \sum (AEP_i \times PoEW_i) + Interface + Gate + Dependency Map + Version History$$

其中:

- `AEP\_i`: 第 i 个工程原子包;
- `PoEW\_i`: 该 AEP 产生的工程工作量证明;
- `Interface`: 该 NEM 对外暴露的能力边界和调用方式;
- `Gate`: 版本验收闸门;
- `Dependency Map`: 它依赖谁、谁依赖它;
- `Version History`: NEM 从 v0.1 到 v1.0 或 v2.0 的演化记录。

更直观地说:

$$NEM = AEP \text{ 积累} + \text{验收} + \text{接口} + \text{版本} + \text{依赖}$$

没有 AEP, NEM 只是空壳。

没有 Gate, NEM 无法证明成熟。

没有 Interface, NEM 无法被其他模块依赖。

没有 Version History, NEM 无法进化。

5. NEM 的七个核心属性

5.1 Composable 可组合

NEM 由多个 AEP 组合而成。每个 AEP 都像一个工程原子, 多个工程原子按照一定结构结合, 形成节点分子。

5.2 Evolvable 可进化

NEM 不以“一次完成”为终点, 而以不断升级为基本存在方式。

一个 NEM 可以从:

v0.1 BASIC → v0.3 STABLE → v0.5 EXTENDED → v1.0 ADOPTED → v2.0 REFACTORED

持续进化。

5.3 Versionable 可版本化

NEM 必须有清晰版本。不同版本代表不同成熟度和可依赖程度。

5.4 Validatable 可验证

NEM 的升级必须经过 Gate。不能凭感觉说“差不多完成了”, 必须通过输出、日志、报告、测试和 PoEW 证明。

5.5 Dependable 可依赖

成熟的 NEM 可以成为其他 NEM 的上游依赖。例如 Runtime NEM 可以被 MRS Benchmark NEM、Sample Library NEM、Report System NEM 依赖。

5.6 Refactorable 可重构

当 NEM 内部结构复杂、旧设计限制新能力时，可以通过一批新的 AEP 进入 v2.0 REFACTORED。

5.7 Replaceable 可替换

如果一个 NEM 的接口清晰，未来可以用更强版本替换旧版本，而不破坏整个工程链。

6. NEM 的版本成熟度模型

NEM 不是“完成 / 未完成”二元状态，而是成熟度状态。

版本	名称	典型 AEP 数量	含义
v0.1	BASIC	3-5	基础可用，形成最小闭环
v0.3	STABLE	8-12	初步稳定，具备状态、日志、恢复、错误分类
v0.5	EXTENDED	12-18	能力增强，开始支持复杂场景
v1.0	ADOPTED	18-25+	主线采纳，可作为下游依赖
v2.0	REFACTORED	不固定	架构重构，形成新一代模块

这里的 AEP 数量不是硬性标准，而是“工程密度”的估计。一个小型 NEM 可能 5 个 AEP 就能进入 v1.0，一个复杂 NEM 可能需要 30 个 AEP 才能稳定。

真正判断 NEM 成熟度的不是 AEP 数量，而是：

能力是否形成闭环；
结果是否可验证；
输出是否可复现；
接口是否清晰；
下游是否可以稳定依赖。

7. NEM 生命周期

一个 NEM 的生命周期建议分为七个阶段：

DRAFT → SEEDED → BASIC → STABLE → ADOPTED → UPGRADED → REFACTORED

7.1 DRAFT 草案期

NEM 还只是一个概念节点。此时需要定义：

- 它为什么存在；
- 它服务哪条工程链；
- 它解决什么能力问题；
- 它的上游和下游是谁。

7.2 SEEDED 种子期

开始投放第一批 AEP。此阶段通常解决最小可运行问题。

7.3 BASIC 基础期

NEM 具备最小闭环，可以被内部使用，但还不一定稳定。

7.4 STABLE 稳定期

NEM 具备日志、状态、失败分类、恢复机制、基础报告和可复现测试。

7.5 ADOPTED 主线采纳期

NEM 成为正式工程链的一部分，可以被其他 NEM 依赖。

7.6 UPGRADED 升级期

在不推翻原结构的前提下，通过新的 AEP 增强能力。

7.7 REFACTORED 重构期

当旧结构限制新能力，NEM 进入重构周期，可能形成 v2.0 或更高版本。

8. NEM 的标准内部结构

一个 NEM 应该有自己的模块目录或注册文件。它不是一个单独的 AEP 包，而是管理多个 AEP 的上层模块。

建议结构：

```
NEM-MT-001_Runtime/
├── nem_manifest.yaml
├── nem_brief.md
├── nem_interface.md
├── nem_dependency_map.yaml
├── nem_version_history.md
├── aep_index.yaml
├── gates/
│   ├── v0.1_basic_gate.md
│   ├── v0.3_stable_gate.md
│   └── v1.0_adopted_gate.md
```

```

├── aeps/
│   ├── MT-001-AEP-001_Runtime_Node_Protocol.zip
│   ├── MT-001-AEP-002_90_Task_Full_Test.zip
│   └── MT-001-AEP-003_State_Machine.zip
├── reports/
│   ├── v0.1_basic_report.md
│   └── v0.3_stable_report.md
└── changelog.md

```

8.1 nem_manifest.yaml

记录 NEM 的身份、版本、状态、上下游依赖和当前成熟度。

8.2 nem_interface.md

定义这个 NEM 对外提供什么能力。没有接口定义的 NEM，很难被下游依赖。

8.3 aep_index.yaml

记录该 NEM 已吸收哪些 AEP，每个 AEP 的状态、PoEW 和对版本升级的贡献。

8.4 gates/

保存不同版本的验收标准。

8.5 reports/

保存 NEM 每次升级后的总结报告。

9. NEM 命名规范

建议使用以下命名：

NEM-<TrackID>-<NodeID>-<ShortName>

例如：

```

NEM-MT-001_Runtime
NEM-MT-002_MRS_Benchmark
NEM-ST-001_MRS_Performance_Profiling

```

AEP 则命名为：

<TrackID>-<NodeID>-AEP-<Number>-<ShortName>.zip

例如：

```

MT-001-AEP-001_Runtime_Node_Protocol.zip
MT-001-AEP-002_90_Task_Full_Test.zip
MT-001-AEP-003_State_Machine.zip

```

这样层级关系会非常清楚：

MT-001 = 主线节点编号

NEM-MT-001 = 节点进化分子
MT-001-AEP-001 = 该 NEM 吸收的第一个工程原子

10. NEM 的 Gate 验收模型

NEM 的升级必须经过 Gate。不同版本有不同 Gate。

10.1 v0.1 BASIC Gate

通过条件:

- 至少完成第一批核心 AEP;
- 形成最小可运行闭环;
- 输出物完整;
- 有基础报告;
- 无关键未解释错误。

10.2 v0.3 STABLE Gate

通过条件:

- 具备状态记录;
- 具备错误分类;
- 具备日志和 summary;
- 具备恢复或重新执行机制;
- 关键流程可复现;
- 支持至少一次中等规模运行或测试。

10.3 v1.0 ADOPTED Gate

通过条件:

- 该 NEM 可以被至少一个下游 NEM 依赖;
- 接口清晰;
- 输出标准稳定;
- 风险记录完整;
- 所有 P0 问题关闭或进入已知限制;
- 形成正式版本报告。

Gate 的意义不是形式主义，而是防止“感觉完成”。

11. NEM 与 Docker / 模块化思想

NEM 与 Docker、软件模块、包管理系统有相似之处。

它们都强调:

封装
版本
依赖
接口
替换
迁移
升级

但 NEM 与 Docker 不同。Docker 主要封装运行环境，而 NEM 封装的是工程能力演化。

Docker 封装环境。
AEP 封装工程做功单元。
NEM 封装节点级工程能力。

NEM 更像 AI 时代的“工程分子模块”。它不是只让代码能运行，而是让一个工程能力可以持续成长、被验证、被依赖、被升级。

12. NEM 在工程链中的位置

完整层级如下：

```
Engineering Organism / 工程生命体
↓
Engineering Chain / 工程链
↓
NEM / 节点进化分子
↓
AEP / 工程原子包
↓
Task Card / 原子指令
↓
PoEW / 工程工作量证明
```

例如 Moodify 可以被表达为：

```
Moodify = 工程生命体
Moodify Runtime Main Track = 工程链
NEM-MT-001 Runtime = 节点进化分子
MT-001-AEP-001 = 工程原子包
Task Card = 原子指令
PoEW = 工程证明
```

这套层级让项目从“聊天推进”变成“工程生长”。

13. NEM 示例：Runtime 节点进化分子

以 Runtime 为例：

NEM-MT-001_Runtime

v0.1 BASIC 目标

目标：让 Runtime 能够完成最小自动运行闭环。

可能需要 5 个 AEP：

```
MT-001-AEP-001 Runtime Node Protocol
MT-001-AEP-002 Runtime 90-task Full Test
MT-001-AEP-003 Basic Logging and Summary
MT-001-AEP-004 Output Directory Standard
MT-001-AEP-005 Basic Gate Report
```

v0.3 STABLE 目标

目标: 让 Runtime 支持稳定运行、失败分类和恢复。

追加 AEP:

MT-001-AEP-006 State Machine
MT-001-AEP-007 Failure Classification
MT-001-AEP-008 Circuit Breaker
MT-001-AEP-009 Timeout and Stalled Detection
MT-001-AEP-010 Resume Protocol

v1.0 ADOPTED 目标

目标: Runtime 成为其他 NEM 可以依赖的正式工程模块。

追加 AEP:

MT-001-AEP-011 6h Endurance Run
MT-001-AEP-012 24h Day Run
MT-001-AEP-013 Runtime Interface Contract
MT-001-AEP-014 Downstream Integration Test
MT-001-AEP-015 Final Adoption Gate

这说明一个 NEM 不是“一次性完成”，而是在多批 AEP 中不断升级。

14. NEM 对 AEP 标准的影响

AEP Standard v0.1 主要定义了“工程原子包”。

NEM 概念锁定后，AEP Standard v0.2 应该增加“分子层”规范:

AEP Layer = 原子层
NEM Layer = 分子层
Chain Layer = 工程链层
Organism Layer = 工程生命体层

v0.2 应新增:

nem_manifest.yaml
nem_interface.md
aep_index.yaml
nem_dependency_map.yaml
version_gate/
nem_version_history.md

这会让 AEP 不只是单个任务外包协议，而进入“模块化工程演化协议”。

15. NEM 的价值

NEM 的价值在于，它解决了 AI 时代工程推进中的一个核心问题:

单个任务可以被 AI 执行，
但复杂工程需要可以持续生长的能力模块。

AEP 解决“怎么让 AI 执行一个标准任务”。

NEM 解决“怎么让多个 AI 执行结果积累成一个可依赖模块”。

没有 NEM，AEP 会变成很多零散工程包。

有了 NEM，AEP 会被组织成持续进化的工程分子。

16. 概念锁定

从本版本开始，建议正式锁定以下概念：

AEP = Atomic Engineering Package / 工程原子包
NEM = Node Evolution Molecule / 节点进化分子
PoEW = Proof of Engineering Work / 工程工作量证明
Gate = Evolution Gate / 进化闸门
Track = Engineering Chain / 工程链
Project = Engineering Organism / 工程生命体

核心句：

AEP 是工程原子，NEM 是节点进化分子。

扩展句：

AEP 将工程任务封装为最小可执行、可验证的工程原子；NEM 则由多个 AEP 组合而成，是可以持续升级、重构和被依赖的节点进化分子。

最终句：

AI 原生工程不是任务列表，而是工程原子组成工程分子，工程分子连接工程链，工程链构成工程生命体。

17. 下一步建议

17.1 写入 AEP GitHub 项目

建议在 AEP Protocol 项目中新增：

docs/NEM_Node_Evolution_Molecule_v0.1.md
docs/NEM_Node_Evolution_Molecule_v0.1.pdf

17.2 增加 NEM 模板

在项目中新增：

```
templates/basic_nem/  
├── nem_manifest.yaml  
├── nem_brief.md  
├── nem_interface.md  
├── aep_index.yaml  
├── nem_dependency_map.yaml  
├── gates/  
└── reports/
```

17.3 以 Moodify Runtime 创建第一个 NEM

第一个真实案例建议为：

NEM-MT-001_Runtime

它将成为 AEP / NEM 理论的第一个工程验证样本。

18. 结语

NEM 的意义，不只是给“节点”换一个名字。

它重新定义了 AI 时代的工程组织方式：

不是用人力连续推进项目，
而是用标准化工程原子不断组合、验证和升级工程分子。

如果 AEP 是 AI 时代的任务外包基础协议，那么 NEM 就是 AI 时代的模块进化基础概念。

AEP 让工程可以被拆分。

NEM 让工程可以被积累。

AEP 让 AI 可以做功。

NEM 让 AI 的做功形成长期能力。

AEP 是工程原子。NEM 是节点进化分子。